Tabla 1. Mecanismos, componentes y propiedades

Qué hace cada mecanismo del programa, qué datos necesita, cuáles deben mantenerse en secreto y cuáles pueden viajar por el canal (que es lo único que ve Eve).

Mecanismo	Entradas	Qué protege (propiedad)	Secreto (no debe viajar)	Público (puede viajar)	Qué produce
SHA-256 (función hash)	Mensaje	Integridad frente a cambios accidentales. No prueba quién lo envió.	Nada: no usa llave	Mensaje y huella	Huella de 32 bytes
SHAKE128 (esponja, salida extensible)	Mensaje y largo de salida deseado	Igual que el hash, pero tú eliges de qué largo sale el resultado	Nada: no usa llave	Mensaje y salida	Huella del largo que se pida
AES-256-CBC (cifrado por bloques)	Mensaje y llave	Confidencialidad: nadie lee el mensaje. No detecta cambios.	La llave	IV (16 bytes al azar) y texto cifrado	Texto cifrado
AES-256-GCM (cifrado por bloques autenticado)	Mensaje, datos asociados y llave	Confidencialidad, integridad y autenticidad (quien lo envió tiene la llave)	La llave	Nonce (12 bytes), datos asociados, texto cifrado y etiqueta	Texto cifrado con etiqueta de 16 bytes
AES-256-CTR (cifrado por flujo)	Mensaje y llave	Confidencialidad. No detecta cambios.	La llave	Nonce y texto cifrado	Texto cifrado del mismo largo que el mensaje
HMAC-SHA256 (código de autenticación)	Mensaje y llave	Integridad y autenticidad. No oculta el mensaje.	La llave	Mensaje y etiqueta	Etiqueta de 32 bytes
Scrypt (derivación de claves)	Contraseña, sal y parámetros de costo	Convierte una contraseña en una llave y hace cara cada conjetura	La contraseña	Sal, parámetros y llave derivada	Llave de 32 bytes
RSA-OAEP (cifrado de clave pública)	Mensaje corto (hasta 190 bytes) y llave pública de Bob	Confidencialidad sin haber compartido un secreto antes. No prueba quién lo envió.	Llave privada de Bob	Llave pública y texto cifrado	Texto cifrado de 256 bytes
RSA-PSS (firma digital)	Mensaje y llave privada de Alice	Integridad, autenticidad y no repudio (Alice no puede negar que firmó)	Llave privada de Alice	Mensaje, firma y llave pública	Firma de 256 bytes
ECDH P-256 (intercambio de claves)	Una llave privada de cada lado	Que Alice y Bob lleguen a la misma llave sin enviarla. No comprueba con quién se habla.	Las llaves privadas y la llave compartida	Los dos valores públicos	Llave compartida de 32 bytes
PKI (certificado X.509 con firma)	Mensaje, certificado y CA de confianza	Comprueba que una llave pública realmente es de quien dice ser	Llaves privadas de la CA y de Alice	Certificado, firma y mensaje	Aceptado o rechazado

Tabla 2. Recursos que consume cada mecanismo

Los tamaños son los de la implementación. Las columnas de tiempo y memoria debes llenarlas con los números que muestra el programa cuando lo ejecutas (Alice y Bob los reportan). Cambian según tu computadora, así que anota los de tu corrida. La memoria medida desde Python no incluye lo que reservan las bibliotecas nativas (OpenSSL), por eso es menor que la real.

Mecanismo	Tamaño de salida	Cómo crece el costo	Tiempo medido (ms)	Memoria Python (KiB)
SHA-256	32 bytes	Crece con el tamaño del mensaje; muy rápido		
SHAKE128	El que se pida	Crece con el mensaje y con el largo de salida		
AES-256-CBC	Múltiplo de 16 bytes (mensaje más relleno)	Rápido; crece con el mensaje		
AES-256-GCM	Mensaje más 16 bytes de etiqueta	Muy rápido; crece con el mensaje		
AES-256-CTR	Igual que el mensaje	Rápido; crece con el mensaje		
HMAC-SHA256	32 bytes de etiqueta	Muy rápido		
Scrypt	32 bytes de llave y 16 de sal	Lento a propósito; gasta mucha memoria real (unos 16 MiB)		
RSA-OAEP	256 bytes	Cifrar es rápido, descifrar es lento; el mensaje es muy limitado		
RSA-PSS	256 bytes de firma	Firmar es lento, verificar es rápido		
ECDH P-256	65 bytes por valor público	Rápido		
PKI	Certificado más firma	Verificar implica comprobar dos firmas; emitir necesita una CA		

Tabla 3. Qué pasa si se usa mal cada mecanismo

Las condiciones que hay que cumplir para conservar la seguridad y la consecuencia de no cumplirlas.

Mecanismo	Condiciones para usarlo bien	Qué pasa si se usa mal
SHA-256	Enviar la huella por un canal que Eve no pueda cambiar. Para contraseñas, usar una derivación de claves.	Eve cambia el mensaje y recalcula la huella, y Bob lo acepta. Con contraseñas, un diccionario las adivina muy rápido.
SHAKE128	Usar una salida de al menos 32 bytes.	Con una salida muy corta es fácil encontrar dos mensajes con la misma huella.
AES-256-CBC	IV nuevo e impredecible en cada mensaje; acompañarlo de una etiqueta.	Eve altera bytes y Bob no lo detecta. Los errores de relleno pueden delatar información.
AES-256-GCM	Nunca repetir el nonce con la misma llave.	Si se repite, se filtra información de los mensajes y se pueden falsificar etiquetas.
AES-256-CTR	Nonce nuevo cada vez; agregar autenticación.	Eve cambia bytes concretos (en la demo cambia 100 por 900) y Bob lo acepta. Repetir el nonce revela los mensajes.
HMAC-SHA256	Llave aleatoria y larga; comparar etiquetas con una función segura; agregar contador contra repeticiones.	Sin contador, Eve reenvía un mensaje válido y Bob lo acepta otra vez. Si la llave se comparte con muchos, cualquiera puede suplantar.
Scrypt	Sal distinta para cada usuario y costo alto de tiempo y memoria.	Con costo bajo o sal repetida, un diccionario adivina contraseñas mucho más rápido.
RSA-OAEP	Llave de 2048 bits o más; solo mensajes cortos; asegurarse de que la llave pública sea la de Bob.	Cualquiera puede cifrar para Bob, así que no prueba quién escribió. Si Eve cambia la llave pública, el mensaje se cifra para ella.
RSA-PSS	Verificar con una llave pública que ya se sabe que es de Alice.	Si se confía en la llave que viaja junto al mensaje, Eve firma con la suya y se hace pasar por Alice.
ECDH P-256	Autenticar los valores públicos con firmas o certificados.	Eve se pone en medio y comparte una llave distinta con cada lado sin que lo noten.
PKI	Confiar solo en CAs conocidas; revisar fechas, nombre y revocación.	Si se acepta un certificado de una CA falsa, Eve se hace pasar por Alice.

Tabla 4. Demostraciones y resultados esperados

Corresponden a la opción 4 del programa. Se hacen con datos de prueba creados por el propio programa.

Escenario	Qué hace Eve	Resultado en el programa	Qué se aprende
Contraseña débil	Prueba un diccionario de 12 palabras contra un hash SHA-256 de prueba	Encuentra la contraseña en pocos intentos	Adivinar una contraseña no es descifrar un hash: el hash no se invierte, se prueban conjeturas
Derivación de contraseñas	Compara intentos contra SHA-256 directo y contra Scrypt	Scrypt es decenas de miles de veces más lento por intento	Los parámetros de costo hacen más caro cada intento del atacante
Cifrado sin autenticación	Cambia un byte del texto cifrado (AES-CTR)	Bob acepta el mensaje alterado	Cifrar no garantiza detectar modificaciones
Cifrado autenticado	Altera el cifrado, los datos asociados o el nonce de AES-GCM	Bob rechaza los tres casos	La verificación debe fallar ante cualquier cambio
HMAC	Cambia el mensaje sin conocer la llave	La etiqueta original ya no sirve y Bob rechaza	Sin la llave no se puede generar otra etiqueta válida
Firma digital	Modifica un mensaje firmado	La firma deja de verificarse	Cualquier cambio invalida la firma
Acuerdo de claves sin autenticación	Sustituye los valores públicos (intermediario simulado)	Eve y Bob terminan con la misma llave y Alice con otra; con firmas, Bob rechaza	Un acuerdo de claves necesita autenticar a los participantes
Repetición de mensajes	Reenvía un mensaje válido	Sin contador Bob lo acepta dos veces; con contador rechaza la segunda	La autenticación necesita controles extra contra repeticiones
PKI	Presenta un certificado de una CA que no es de confianza	Bob lo rechaza	El receptor debe rechazar certificados que no vienen de una CA confiable